

Понял. Формулирую **задачи максимум** на сегодня (и ближайшие дни), разбиваю на порядок.

Приоритет 1. Доводка визуального отображения формы

Задача	Что сделать
Заголовок (caption)	У формы есть <code>windowTitle</code> , но в дизайнера он не отображается. Добавить в <code>FormBackgroundItem</code> строку заголовка с возможностью редактирования в свойствах
Кнопки окна	Свернуть, развернуть, закрыть — добавить как часть заголовка (пока без логики, просто визуально)
Тень/объём	Добавить <code>QGraphicsDropShadowEffect</code> для формы
Сетка на форме	Показать сетку (шаг 10px) для удобного позиционирования контролов. Включается/выключается чекбоксом в тулбаре

Приоритет 2. Три закладки в панели свойств

Закладка	Содержимое
Свойства	Все свойства формы или контрола (существующая таблица, но расширенная)
Методы	Список методов формы/контрола (<code>on_open</code> , <code>on_close</code> , <code>on_save</code> , <code>on_click</code> ...). При двойном клике — открыть редактор кода
События	Привязка событий контролов к методам (например, <code>btn_save.on_click → save_form()</code>)
Реализация: Заменяем текущую панель свойств на <code>QTabWidget</code> с тремя вкладками.	

Приоритет 3. Системные и контекстные свойства формы

Переносим сюда всё, что было на вкладке «Свойства формы»:

Свойство	Тип	Назначение
<code>sname</code>	<code>str</code>	Человеческое название формы
<code>calias</code>	<code>str</code>	Системный алиас (уникальный)
<code>identitytypeid</code>	<code>int</code>	ID якорной сущности (опционально)
<code>mstate_json</code>	<code>text</code>	JSON-состояние формы

Добавляем контекстные свойства (для использования в L3):

Свойство	Тип	Назначение
<code>caller_form</code>	<code>str</code>	Алиас формы, которая вызвала эту
<code>entity_instance_id</code>	<code>int</code>	ID экземпляра сущности, который редактируем
<code>entity_alias</code>	<code>str</code>	Алиас сущности (если форма привязана)
<code>form_index</code>	<code>int</code>	Индекс в коллекции открытых форм
<code>open_params</code>	<code>text</code>	JSON параметров, переданных при открытии
<code>is_modal</code>	<code>bool</code>	Модальная / немодальная
<code>is_readonly</code>	<code>bool</code>	Только для чтения

После этого — удаляем вкладку «Свойства формы» из `FormPropertiesDialog`.

Приоритет 4. Режимы формы (грид / едитная)

Если форма привязана к сущности (`identitytypeid` не пуст), пользователь при создании формы выбирает тип:

Тип	Описание
Грид	Список записей (таблица), редактирование — в отдельной форме или в диалоге
Едитная	Карточка одной записи (редактирование полей)

Нужен визард (мастер) при создании формы:

- Шаг 1. Выбрать сущность
 - Шаг 2. Выбрать тип (грид / едитная)
 - Шаг 3. Выбрать поля для отображения (для грида — колонки, для едитной — расположение на форме)
 - Шаг 4. Сгенерировать форму автоматически
-

Приоритет 5. `DataBinding` и передача данных на L2

Как должно работать:

1. Форма получает `entity_instance_id` и `entity_alias` при открытии.
2. Загружает данные через L2 (вызов `get_item`).
3. Заполняет контролы (`DataBinding`).
4. При изменении данных — помечает контролы как `dirty`.
5. При сохранении — собирает изменённые данные, отправляет на L2 через `save_item`.
6. L2 выполняет транзакцию (создаёт новую версию или обновляет).

Что нужно в конфигураторе:

- Для каждого контрола, привязанного к полю сущности, в свойствах указывать `field` (имя поля в ext-таблице).
 - В форме хранить список привязок: `{control_alias: field_name}`.
-

Порядок выполнения (что делаем первым)

Я предлагаю такой порядок:

1. Тень, заголовок, кнопки окна, сетка — быстро, визуально приятно.

2. **Три закладки в панели свойств** — рефакторинг, но нужный.
3. **Перенос системных свойств** — объединяем всё в одной панели.
4. **Удаляем лишнюю вкладку «Свойства формы».**
5. **Контекстные свойства** — добавляем.
6. **Режимы формы и визард** — отдельная большая задача.
7. **DataBinding** — следующая итерация.

С какого пункта начинаем? Я бы начал с **пункта 1 (тень, заголовок, кнопки, сетка)** — это даст быстрый визуальный результат и поднимет настроение.

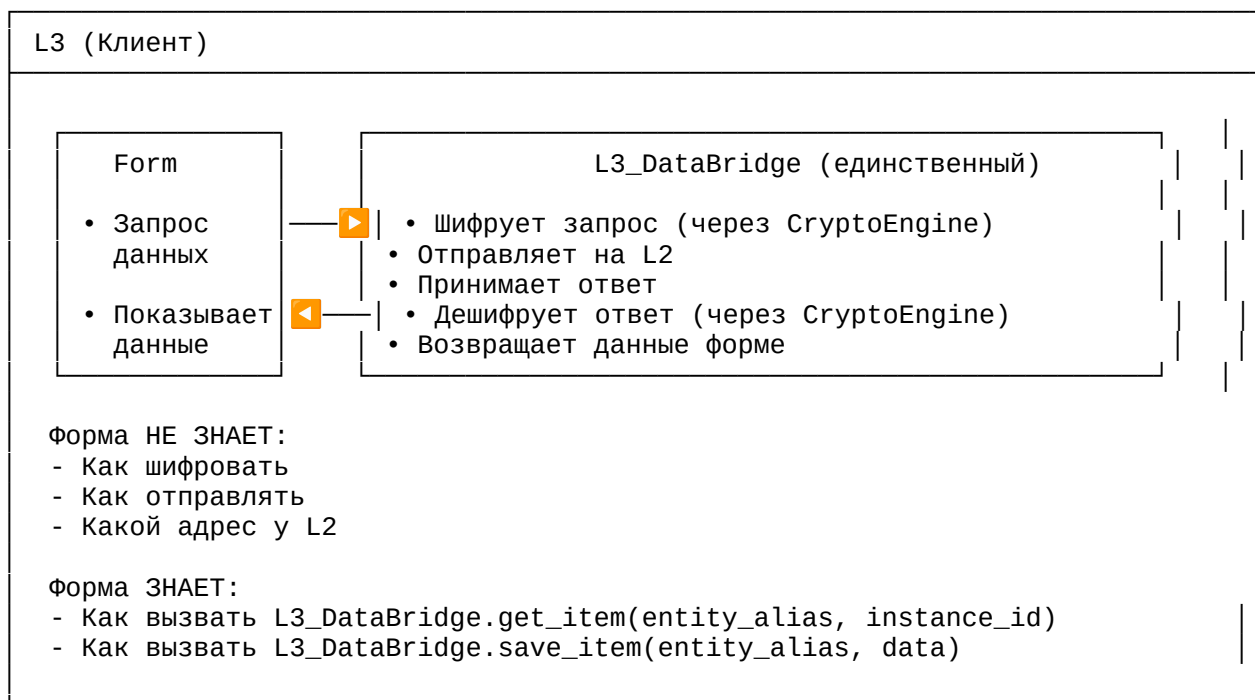
Жду команды.

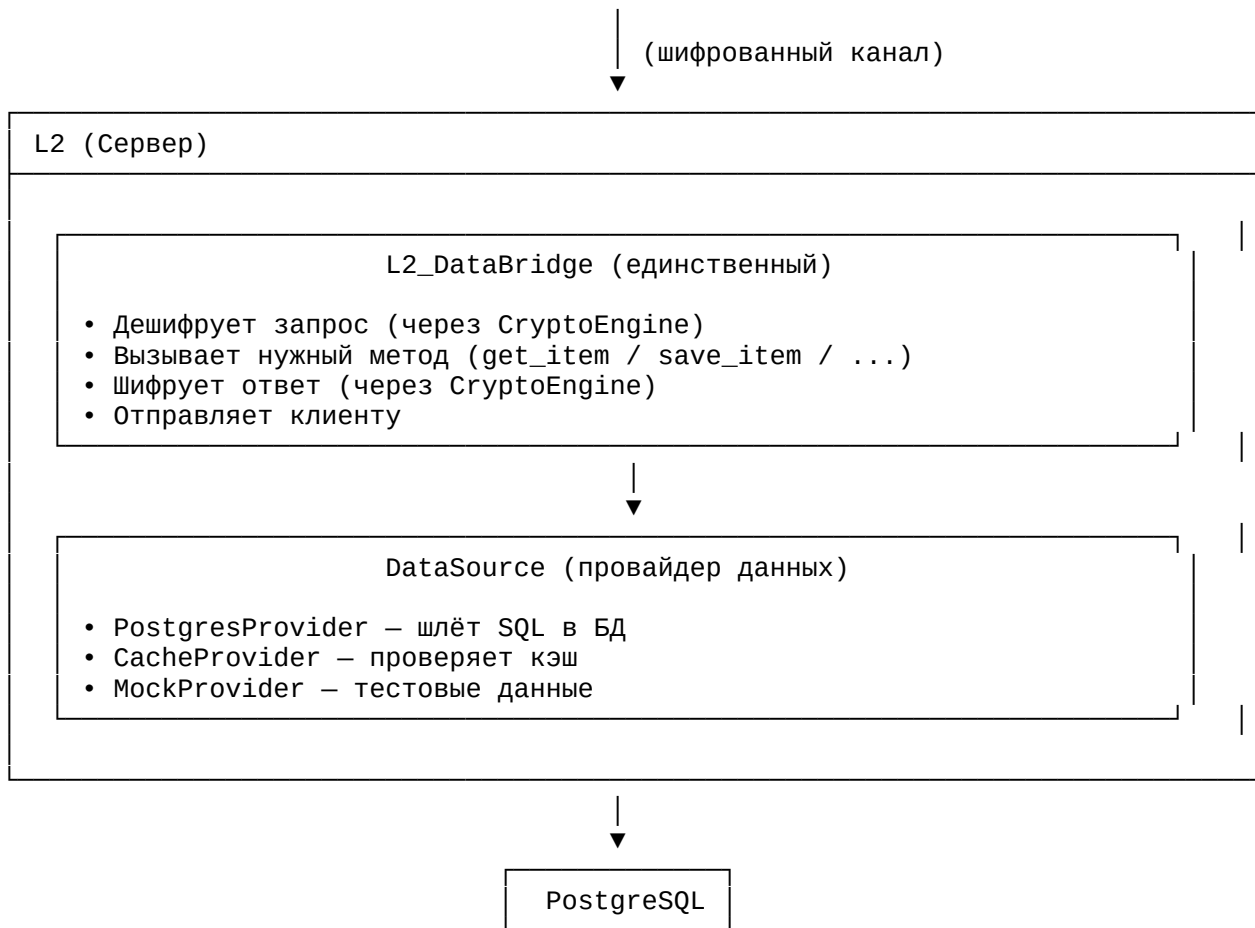
Приоритет 5. DataBinding и передача данных на L2 Как должно работать: Форма получает `entity_instance_id` и `entity_alias` при открытии. Загружает данные через L2 (вызов `get_item`).-----стоп стоп. данные форма загружает только через проводника связи со вторым слоем. сама форма со 2 слоя ничего не порлучает. она направляет запрос объекту связи со слоем 2. а вот он дергает 2 слой(используя встроенный объект шифрации /дешифрации - кроме этого объекта НИ ОДИН доугой ни шифровать ни дешифровывать не умеет, в этом соль)

Поправляю. Ты абсолютно прав.

Исправленная схема DataBinding

text





Ключевые принципы (золотые правила)

Правило	Кто соблюдает
Шифровать/дешифровать умеет только CryptoEngine	Весь L3 и L2
Отправлять/принимать на L2 умеет только L3_DataBridge	Весь L3
Принимать от L3 и отправлять обратно умеет только L2_DataBridge	Весь L2
Форма не знает про L2 и про шифрование	Все формы
Форма знает только L3_DataBridge	Все формы

Как форма загружает данные

python

```
# В методе on_open формы
def on_open(self, params):
    entity_alias = params.get('entity_alias')
    instance_id = params.get('instance_id')

    if instance_id:
        # Форма просит DataBridge дать данные
        data = self.data_bridge.get_item(entity_alias, instance_id)
        # DataBridge сам шифрует, отправляет, дешифрует ответ
        # Форма получает готовый словарь
```

```
# Заполняем контролы
for control_alias, field_name in self.bindings.items():
    control = self.get_control(control_alias)
    if control and field_name in data:
        control.value = data[field_name]
```

Как форма сохраняет данные

python

```
# При нажатии кнопки "Сохранить"
def on_save(self):
    # Собираем данные из контролов
    data = {}
    for control_alias, field_name in self.bindings.items():
        control = self.get_control(control_alias)
        if control:
            data[field_name] = control.value

    # Форма просит DataBridge сохранить
    result = self.data_bridge.save_item(
        entity_alias=self.entity_alias,
        instance_id=self.instance_id,
        data=data,
        version_id=self.version_id
    )
    # DataBridge сам шифрует, отправляет, получает ответ

    if result.success:
        self.version_id = result.new_version_id
        # Обновляем original_value у всех контролов
        self.reset_dirty()
```

Что нужно в конфигураторе для DataBinding

При настройке формы в конфигураторе, для каждого контрола (кроме кнопок, панелей и т.д.) указываем:

Свойство	Назначение
binding_field	Имя поля в ext-таблице (например, cname, clastname)
binding_entity	Алиас сущности (если отличается от основной формы)

Пример для Reference-контрола:

json

```
{
    "calias": "person_id",
    "binding_field": "person_id",
    "binding_entity": "person",
    "display_field": "cname"
}
```

Итог

- Форма не знает про L2 и шифрование.
- Форма знает только L3_DataBridge.
- L3_DataBridge — единственный, кто умеет отправлять на L2 и шифровать.
- L2_DataBridge — единственный, кто принимает от L3.
- CryptoEngine — единственный, кто умеет шифровать/дешифровать.

Так понятно? Если да — продолжаем проектировать конфигуратор с учётом этих правил.

да

Отлично. Двигаемся дальше.

Теперь, когда мы зафиксировали **железное правило** (только L3_DataBridge общается с L2, только CryptoEngine шифрует), возвращаемся к **задачам конфигуратора**.

Что сегодня делаем (порядок)

Я предлагаю такой порядок, чтобы быстро получить видимый результат:

1. Визуальные улучшения формы в дизайнера

- Заголовок окна (caption)
- Кнопки свернуть/развернуть/закрыть (пока декоративные)
- Тень (объём)
- Сетка для позиционирования

2. Три закладки в панели свойств

- Заменяем текущую панель на QTabWidget
- Вкладка «Свойства» — существующая таблица
- Вкладка «Методы» — список методов с кнопкой «Редактировать»
- Вкладка «События» — привязка событий контролов

3. Перенос системных свойств

- cname, calias, ientitytypeid, mstate_json — добавляем в панель свойств (вкладка «Свойства»)
- Удаляем вкладку «Свойства формы» из FormPropertiesDialog

4. Контекстные свойства

- Добавляем в панель свойств: `caller_form`, `entity_instance_id`, `entity_alias`, `form_index`, `open_params`, `is_modal`, `is_readonly`

5. DataBinding (начало)

- В свойствах контрола добавляем поле `binding_field`
 - Форма при загрузке будет использовать эту привязку
-

С чего начнём?

Я предлагаю начать с пункта 1 (визуальные улучшения) — это быстро, приятно и видно сразу.

После этого — пункт 3 (три закладки) — рефакторинг, но нужный.

Как скажешь? Начинаем с визуала или сразу лезем в закладки?